

MODULE 19 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Jobs in production

Two sets of permissions, the run-as identity, parameters that actually reach your code, tags and cost.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 19 of 36

A job has **two separate permission systems**. One controls who can touch the job. The other controls what the job itself can touch.

Job privileges

LEVEL	WHAT IT ALLOWS
CAN VIEW	View the job configuration and monitor runs
CAN MANAGE RUN	Trigger, stop and restart runs
CAN MANAGE	Full administrative access: edit configuration, tasks and scheduling
IS OWNER	The creator by default. A job cannot have more than one owner.

ON THE EXAM

Each level includes the grants below it. Workspace admins are granted CAN MANAGE. Only grant CAN MANAGE or IS OWNER to people trusted to modify production code.

Anyone with **CAN MANAGE RUN** or higher can override job parameter values when running manually. A default parameter value is **not a security control**.

The run-as identity

- ◆ **Determines what the job can access** — the data, the secrets, the external locations.
- ◆ **Use a service principal for production jobs**, not a named person.
- ◆ **A job running as a person breaks when they leave**, or when their access changes for an unrelated reason.
- ◆ **It also lets you restrict write access to production data**, granting it to the service principal rather than to every engineer.

ON THE EXAM

If a job runs as a user, that user needs every permission the job needs — including write access to production tables.

Four things called parameters

CONCEPT	WHAT IT IS
Job parameter	A key-value pair defined at the job level and pushed down to tasks
Task parameter	A key-value pair or JSON array defined on one task
Dynamic value reference	Syntax for referencing job conditions, metadata and parameters
Task value	A value produced during a run, for later tasks to read

ON THE EXAM

A notebook widget is not a fifth store. It is the interface through which notebook code reads whichever value was delivered.

The task type changes how parameters arrive

- ◆ **A notebook task receives key-value parameters.** Job parameters are pushed down automatically.
- ◆ **A Python script or JAR task receives an ordered JSON array.** Job parameters are not pushed down the same way.
- ◆ **Those tasks reference job parameters explicitly,** using dynamic value references in their task parameters.
- ◆ **Job parameter defaults are resolved when the run starts,** and can be overridden for that run.

ON THE EXAM

'I set a job parameter and the script cannot see it' is almost always this. Key-value tasks get it free; array tasks must ask.

Dynamic value references

- ◆ **Syntax for injecting values you cannot know when writing the job**, resolved at run time.
- ◆ **Job and run metadata**: the job id, the run id, the scheduled start time.
- ◆ **Other parameters and task values**, so a task parameter can be built from a job parameter.
- ◆ **The start time reference is how a task learns which date it is processing**, rather than reading the clock itself.

ON THE EXAM

A task that calls `current_date()` processes today. A task given the scheduled start time processes the right day, even on a re-run three days later.

Tags

- ◆ **Labels or key-value attributes attached to a job**, edited in the Job details pane.
- ◆ **They flow through to billing**, which is how compute cost gets attributed to a team, project or cost centre.
- ◆ **Without them, the bill is one undifferentiated number**, and nobody can act on it.
- ◆ **Enforce them with a cluster policy**, as module 2 covered, so an untagged job cannot be created.

ON THE EXAM

Tagging is the difference between knowing you spent a lot and knowing which team spent it. That is what makes it actionable.

Notifications that catch the quiet failures

- ◆ **On failure** is the obvious one, and it misses everything that does not fail.
- ◆ **Duration thresholds** alert when a run takes longer than expected, catching gradual degradation.
- ◆ **Per-task notifications** matter because of the leaf task rule from module 17.
- ◆ **A job that succeeds while doing nothing useful sends no notification at all.**

ON THE EXAM

Module 17's scenario is the case in point: a handled failure is not a job failure, so job-level alerting stays silent.

Compute governance

- ◆ **Serverless compute for jobs and SQL warehouses always use Unity Catalog.**
- ◆ **For classic compute, Databricks recommends standard access mode** for supported workloads.
- ◆ **Use dedicated access mode only when required**, as module 2 set out.
- ◆ **Unity Catalog governance requires a supported compute configuration.** No isolation shared is not one.

ON THE EXAM

If a scenario needs governed data access from a job, the compute configuration is as much a part of the answer as the grants are.

What lives in the Job details pane

SETTING	WHY IT MATTERS
Schedule or trigger	Module 18. Time-based or data-driven.
Job parameters	Pushed down to key-value tasks; overridable at run time
Compute configuration	Cost and governance both follow from this
Tags	Cost attribution
Notifications and duration thresholds	Whether anyone finds out
Maximum concurrent runs	What happens when a trigger fires mid-run
Git settings	Which branch or tag the job runs from
Permissions	Who can view, run and manage

ON THE EXAM

Eight settings, and only the first two are about what the job does. The other six are about operating it.

Tiebreakers from this module

WHEN A QUESTION SAYS

Someone ran the job with a different parameter

An on-call engineer must rerun but not edit

The job broke when an engineer left

A Python script cannot see a job parameter

A repaired run processed the wrong date

Nobody can tell which team the bill belongs to

The job runs slower every month, but never fails

REACH FOR

CAN MANAGE RUN allows it. Defaults are not security.

CAN MANAGE RUN

It ran as a person. Use a service principal.

Array tasks need a dynamic value reference

It read the clock instead of the scheduled start time

Tags, enforced by a cluster policy

A duration threshold notification

ON THE EXAM

Seven lines. The first is the one most likely to be phrased as a security question rather than a jobs question.

A job takes a parameter named `target_catalog`, with a default value of `dev_catalog`. The team relies on this default to ensure that only their deployment pipeline, which passes `prod_catalog` explicitly, can write to production.

An analyst with CAN MANAGE RUN on the job uses Run now with different parameters and sets `target_catalog` to `prod_catalog`. Production data is overwritten.

What went wrong?

- A A defect; parameter overrides should require CAN MANAGE
- B A default parameter value is not a security control; CAN MANAGE RUN permits overrides, so job permissions must restrict this
- C The analyst must have been granted CAN MANAGE by mistake
- D Job parameters cannot be overridden; the deployment pipeline must have been misconfigured

◆ PAUSE AND DECIDE

What went wrong?

- A

A defect requiring CAN MANAGE

Documented behaviour, not a defect. CAN MANAGE RUN or higher can override parameters for a manual run, by design.
- B

Defaults are not a security control

Correct. The docs say so directly. The only protection is restricting who holds CAN MANAGE RUN on the job.
- C

The analyst had CAN MANAGE by mistake

CAN MANAGE RUN is sufficient. No migrant is needed to explain this, and the scenario states the permission held.
- D

Parameters cannot be overridden

They can, both through Run now with different parameters and through the REST API.

THE SIGNAL

The team built a control out of a convenience. Defaults exist so the job runs sensibly, not to stop anyone changing them.

A production job has run nightly for two years. It was created by an engineer who left the company last month. Her account was deprovisioned yesterday.

The job now fails immediately with a permissions error on the source tables. The on-call engineer has full access to those tables and can query them without difficulty.

What is the cause and the fix?

- ☐ A The job runs as the departed engineer; change the run-as identity to a service principal
- ☐ B Grant the on-call engineer CAN MANAGE on the job
- ☐ C Re-grant SELECT on the source tables to the departed engineer's account
- ☐ D The tables were moved to a different catalog when the account was deprovisioned

◆ PAUSE AND DECIDE

What is the cause and the fix?

- A

 It runs as the departed engineer; move to a service principal
Correct. The run-as identity determines data access. Deprovisioning that account removed the job's ability to read.
- B

 Grant the on-call engineer CAN MANAGE
Job privileges are a separate system from run-as. His access changes who can edit the job, not what the job can read.
- C

 Re-grant SELECT to the departed account
Reviving a deprovisioned account's access to keep production running is the opposite of the correct governance answer.
- D

 The tables were moved
Deprovisioning an account does not move tables. It invents a cause when the timing already points at the account.

THE SIGNAL

The on-call engineer's own access is a red herring, and a deliberate one. His permissions have nothing to do with the job's.

A job's first task is a notebook that reads a job parameter called `run_date` and works correctly. Its second task is a Python script task that needs the same value.

The script receives nothing for `run_date` and fails on a missing argument. The job parameter is definitely set, and the notebook task proves it.

Why, and what is needed?

- A Job parameters only apply to the first task in a job
- B A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference
- C The script must call `dbutils.widgets.get`, as the notebook does
- D Job parameters must be redefined as task parameters on every task that needs them

◆ PAUSE AND DECIDE

Why, and what is needed?

- A

 Only the first task gets them
Invented rule. Job parameters are pushed to every task that uses key-value parameters, regardless of position.
- B

 Array tasks need an explicit dynamic value reference
Correct. Job parameters are pushed down to key-value tasks. A Python script task takes an ordered JSON array, so it must reference the parameter by name.
- C

 Call `dbutils.widgets.get` in the script
Widgets are the notebook interface for reading a delivered value. They do not deliver anything to a script task.
- D

 Redefine as task parameters everywhere
Would work, and it duplicates the value on every task and defeats the point of defining it once at job level.

THE SIGNAL

The difference is the task type, not the parameter. Key-value tasks get the push-down; array tasks must ask by name.

A daily job processes the previous day's partition. The notebook computes the date itself with `current_date()` minus one day. This has worked for a year.

A run fails on Friday. Nobody notices until Monday, when an engineer repairs the failed run. It succeeds. The following week, reconciliation shows Friday's data is still missing, and Sunday's has been processed twice.

What is the design fault?

- ☐ A The repair should have been a fresh run rather than a repair
- ☐ B The task derives its date from the clock, so a Monday repair processed Sunday; it should use the scheduled start time
- ☐ C The job needs a retry policy so failures are handled the same night
- ☐ D `current_date()` returns UTC, and the job runs in a different time zone

◆ PAUSE AND DECIDE

What is the design fault?

- A

 A fresh run instead of a repair
A fresh run on Monday would have processed Sunday too. The problem is where the date comes from, not which button was pressed.
- B

 It reads the clock, not the scheduled start time
Correct. A repair runs on Monday, so `current_date()` minus one gives Sunday. The run's own scheduled start time would have given Friday.
- C

 A retry policy
Retries handle failures the same night, which helps. It does not fix a task that will always process the wrong day whenever it runs late.
- D

 A time zone problem
A time zone offset shifts dates by hours, not by three days. It also would not explain a duplicate.

THE SIGNAL

Two symptoms confirm it: Friday missing and Sunday duplicated. That is one run processing the wrong day, not a run that failed to happen.

A platform team is asked to reduce Databricks spend by 20%. The account bill is large but arrives as a single figure. There are around 400 jobs across eleven teams, and no job carries any tags.

The finance director wants each team told what they are spending so they can act.

What should the platform team do first?

- ☐ A Reduce all cluster sizes by 20% across the workspace
- ☐ B Introduce tags on jobs and enforce them with a cluster policy, then attribute cost using the jobs and billing system tables
- ☐ C Move every job to serverless compute
- ☐ D Set aggressive auto-termination on every cluster

◆ PAUSE AND DECIDE

What should the platform team do first?

- A

 Cut all cluster sizes by 20%
Applies a blunt change to 400 jobs blind. Some will fail, some will spill, and nobody can tell which mattered.
- B

 Tag, enforce, then attribute with system tables
Correct. The stated requirement is that each team be told what they spend, which is impossible without attribution. Measure before cutting.
- C

 Move everything to serverless
Often reduces cost, and it is a blind change across 400 jobs including JAR and Spark Submit tasks that cannot use it.
- D

 Aggressive auto-termination everywhere
A real saving on idle clusters, and it does nothing for the stated requirement of telling each team their spend.

THE SIGNAL

The requirement is not only to spend less. It is that each team be told what they spend, which is a measurement problem before it is a cost one.

Two permission systems, one identity, and a default that protects nothing. **Most production job failures are about who, not about what.**